

Kiến trúc SOA & Microservices

Đặng Ngọc Hùng

Mục lục

11. Quan sát Hệ thống — Observability & Distributed Tracing	4
11.1. Ba trụ cột của Observability	5
11.1.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”	5
11.1.2. Ba trụ cột	5
11.1.3. Observability vs Monitoring	7
11.1.4. Observability Maturity Model	7
11.2. Logging — Từ Console đến Centralized Aggregation	8
11.2.1. Vấn đề: logs phân tán, không có context	8
11.2.2. Structured Logging — Từ text sang JSON	9
11.2.3. Centralized Log Aggregation	9
11.2.4. Correlation IDs — Kỹ thuật quan trọng nhất	10
11.3. Chi tiết về Distributed Tracing	11
11.3.1. Vấn đề: request chậm — chậm ở đâu?	11
11.3.2. Distributed Tracing — Tự động đo timing	11
11.3.3. Concepts: Trace, Span, Context Propagation	12
11.3.4. Sampling Strategies	12
11.3.5. Tracing trong Spring Boot 3.x	13
11.3.6. Kiến trúc triển khai Observability Pipeline	14
11.4. Metrics, SLI/SLO, và Alerting Strategy	14
11.4.1. Metrics — Đo lường hành vi theo thời gian	14
11.4.2. Bốn loại Metrics cơ bản của Prometheus	15
11.4.3. Hai framework đo lường: RED và USE	16
11.4.4. SLI/SLO/SLA — Từ metrics đến cam kết chất lượng	16
11.4.5. Alerting Strategy	17
11.5. Error Handling Strategy — Nhặt quán xuyên Services	18
11.5.1. Nguyên tắc thiết kế Error Handling	18
11.6. Health Checks và Service Discovery	19
11.6.1. Health Checks — Nền tảng cho tự động hóa	19
11.7. User Activity Tracking — Observability ở mức Business	20
11.8. Chaos Engineering — Kiểm tra Resilience chủ động	21
11.8.1. Industry Case Study: Netflix Simian Army	22
11.9. Case Study: KBM	23
11.9.1. Hiện trạng — Đánh giá theo Maturity Model	23
11.9.2. Phân tích theo business context	24
11.10. Tổng kết	26
11.11. Đọc thêm	26
Tài liệu tham khảo	27

11.

CHƯƠNG 11.

Quan sát Hệ thống – Observability & Distributed Tracing

“Observability is not just monitoring, it’s the ability to ask new questions about your system’s behavior without deploying new code.”
— Charity Majors (tổng hợp từ observability community)

MỤC TIÊU HỌC TẬP

- Hiểu tại sao observability là **điều kiện tiên quyết** để vận hành microservices — không chỉ là “nice-to-have”
- Phân biệt **observability vs monitoring** và khi nào cần gì
- Nắm vững ba trụ cột: **Logs Metrics Traces** và cách chúng bổ trợ nhau
- Thiết kế chiến lược **centralized logging** với structured logs và correlation IDs
- Hiểu **distributed tracing** và các khái niệm trace, span, context propagation
- Nắm framework đo lường: **SLI/SLO/SLA** và chiến lược alerting
- Xây dựng **error handling strategy** nhất quán xuyên services
- Phân tích observability gaps trong KBM và đề xuất migration path



Ghi chú Chương này tập trung vào nguyên tắc và thiết kế observability độc lập với hạ tầng triển khai cụ thể. Sau khi đọc Chương 12 (Triển khai & DevOps), người đọc sẽ thấy rõ hơn cách các công cụ observability được tích hợp vào CI/CD pipeline và container runtime.

Để vận hành hệ thống phân tán, các nhóm phát triển cần chuyển từ tư duy giám sát thụ động (Monitoring - hệ thống có đang hoạt động bình thường không?) sang khả năng quan sát chủ động (Observability - tại sao hệ thống chạy chậm?).

11.1. Ba trụ cột của Observability

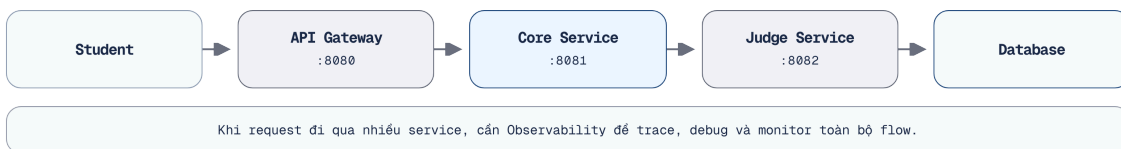
Để đảm bảo hệ thống LMS hoạt động trơn tru, chúng ta cần xây dựng cơ chế giám sát dựa trên ba trụ cột cốt lõi, giúp định vị chính xác mọi sự cố.

11.1.1. Vấn đề: “hệ thống lỗi nhưng không biết lỗi ở đâu”

Trong monolith, khi xảy ra lỗi: chỉ cần mở log file duy nhất, tìm exception và đọc stack trace là có thể định vị được vấn đề. Toàn bộ request chạy trong cùng process, cùng log file, cùng database.

Trong kiến trúc microservices, một yêu cầu (request) từ người dùng có thể đi qua nhiều dịch vụ, mỗi dịch vụ có nhật ký hệ thống (log), cơ sở dữ liệu và chu trình triển khai (deploy cycle) độc lập. Khi một yêu cầu gặp lỗi (fail):

- Service nào gây lỗi? Gateway? Core? Judge? Auth?
- Lỗi xảy ra ở bước nào trong chuỗi gọi?
- Latency 5 giây — do network, do database query, hay do Kafka consumer lag?

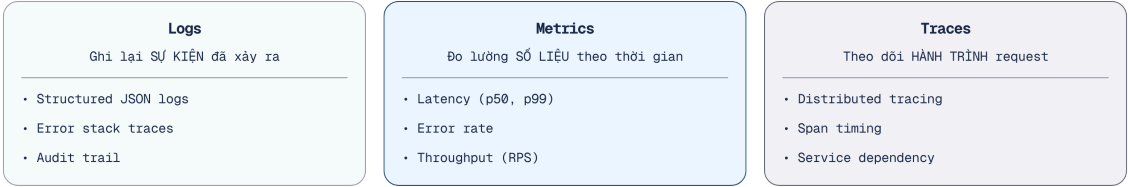


Hình 11.1: Request flow qua 5+ services — lỗi xảy ra ở đâu?

Newman trong [4a, Ch.8] gọi đây là thách thức lớn nhất khi vận hành microservices: **sự phức tạp không nằm ở code, mà nằm ở tương tác giữa các services**. Trong monolith, lỗi thường có stack trace rõ ràng — một file, một method. Trong microservices, lỗi có thể xuất hiện ở dịch vụ B do dịch vụ A gửi dữ liệu sai, hoặc do thông điệp (message) Kafka bị trễ — lúc này, không có dấu vết ngăn xếp (stack trace) nào có thể ghi nhận được nguyên nhân.

11.1.2. Ba trụ cột

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đều mô tả observability qua **ba trụ cột** (three pillars):



Hình 11.2: Ba trụ cột của Observability — Logs, Metrics, Traces

11.2 minh họa ba trụ cột ở mức tổng quan: mỗi trụ cột phục vụ một loại câu hỏi khác nhau khi debug hệ thống phân tán. Bảng dưới đây mô tả chi tiết vai trò, ví dụ cụ thể trong KBM, và các công cụ phổ biến cho từng trụ cột — giúp lập trình viên hình dung cách áp dụng ngay vào hệ thống thực tế.

Trụ cột	Trả lời câu hỏi	Ví dụ trong KBM	Công cụ phổ biến
Logs	“Chuyện gì đã xảy ra?”	ERROR: SQL execution timeout for submission 123	ELK Stack, Loki, Fluentd
Metrics	“Hệ thống đang hoạt động thế nào?”	judge_execution_time_p99 = 4.2s	Prometheus, Grafana, Micrometer
Traces	“Request đi qua những đâu?”	User → Gateway → Core → Judge → MySQL (total: 3.1s)	Jaeger, Grafana Tempo, Zipkin (Dùng chuẩn OpenTelemetry OTLP)

Bảng 11.1: Vai trò của ba trụ cột Observability

M.E.L.T và Events

Trong công nghiệp, thuật ngữ **M.E.L.T** thường xuyên được sử dụng (phổ biến bởi New Relic, Datadog), trong đó chữ **E - Events** được nâng lên thành một trụ cột thứ 4. Tuy nhiên trong thiết kế phân tán, Events thường thuộc về ngữ nghĩa nghiệp vụ (Business Domain) và được tách biệt để truyền tải qua Message Broker (Kafka) thay vì đi chung đường ống giám sát kỹ thuật.

Ba trụ cột **kết hợp với nhau**: metrics cho biết *hệ thống đang có vấn đề* (thời gian phản hồi tăng), traces chỉ ra *vấn đề nằm ở đâu* (Dịch vụ Judge chiếm 90% thời gian xử lý), logs giải thích nguyên nhân *tại sao* (truy vấn SQL bị quá hạn do thiếu chỉ mục - missing index). Thiếu bất kỳ trụ cột nào, hai thành phần còn lại cũng mất đáng kể giá trị.

11.1.3. Observability vs Monitoring

	Monitoring	Observability
Mục đích	Phát hiện vấn đề đã biết	Khám phá vấn đề chưa biết
Cách tiếp cận	Đặt alerts cho metrics cụ thể	Explore data để hiểu hành vi hệ thống
Ví dụ	“CPU > 80% → alert”	“Tại sao latency tăng 3x ở 2PM mỗi thứ 3?”
Dữ liệu	Pre-defined dashboards	Logs + Metrics + Traces (kết hợp ad-hoc)
Khi nào đủ	Hệ thống đơn giản, failure modes biết trước	Hệ thống phân tán, failure modes không dự đoán được

Bảng 11.2: Observability vs Monitoring

Monitoring trả lời: “Có vấn đề không?” Observability trả lời: “Tại sao có vấn đề này, và vấn đề này khác gì so với lần trước?” Với monolith, monitoring thường đủ. Với microservices — nơi failure modes phức tạp và không dự đoán được — observability là yêu cầu bắt buộc.

! Observability ≠ Monitoring

“Monitoring tells you *that* something is wrong. Observability lets you ask *why* Monitoring is a subset of observability — necessary but not sufficient.”

— Tổng hợp từ Charity Majors và cộng đồng observability

Ẩn dụ LMS: Camera giám sát vs Hệ thống Quản trị

Monitoring – Giống như bác bảo vệ nhìn vào màn hình **camera giám sát** và thấy sinh viên đang ùn tắc trước thang máy nhà A. Bác chỉ biết: *Hệ thống đang có vấn đề.*

Observability – Bác bảo vệ có hệ thống liên kết camera với lịch học, thẻ quẹt sinh viên. Tra cứu nhanh, bác phát hiện ra: *Có 2 lớp học bù đột xuất được xếp trùng giờ ở cùng tầng 9.* Bác biết chính xác: *Tại sao có vấn đề và root cause là gì.*

11.1.4. Observability Maturity Model

Không phải hệ thống nào cũng cần full observability stack ngay từ đầu. Maturity model giúp xác định đang ở đâu và cần đến đâu.

Level	Mô tả	Đặc điểm	Phù hợp với
1 — Reactive	Debug khi người dùng báo lỗi	Console logs, không có cảnh báo tự động, điều tra thủ công	Dự án cá nhân, bản mẫu thử nghiệm
2 — Basic monitoring	Dashboards cơ bản, alerts thô	Health checks, CPU/memory metrics, log tập trung	Nhóm nhỏ, hệ thống ít dịch vụ
3 — Proactive	Phát hiện vấn đề trước khi người dùng gặp phải	Structured logs, correlation IDs, custom metrics, alerting	Nhóm quy mô vừa, nhiều dịch vụ
4 — Observability	Hỏi câu hỏi mới không cần code mới	Distributed tracing, SLI/SLO, high-cardinality data	Nhóm quy mô lớn, hệ thống phức tạp
5 — Predictive	Dự đoán vấn đề trước khi xảy ra	ML-based anomaly detection, capacity planning	Enterprise, mission-critical

Bảng 11.3: Observability Maturity Model

Nguồn: Tổng hợp từ Richardson [2a, Ch.11], Newman [4a, Ch.8], và Charity Majors, Liz Fong-Jones, George Miranda, Observability Engineering (O'Reilly, 2022). Maturity levels phản ánh kiến thức cộng đồng observability — không theo một framework độc quyền nào.

Với LMS — hệ thống giáo dục quy mô trung bình — mục tiêu hợp lý là **Level 3** (Proactive), với khả năng mở rộng lên Level 4 khi cần.

11.2. Logging — Từ Console đến Centralized Aggregation

Trong kiến trúc microservices, một request từ người dùng đi qua nhiều dịch vụ. Nếu logs bị phân tán ở các máy chủ khác nhau, việc truy vết lỗi sẽ trở nên vô cùng khó khăn.

11.2.1. Vấn đề: logs phân tán, không có context

Khi hệ thống có 7+ services chạy đồng thời, mỗi service ghi log vào container riêng. Để truy vết một yêu cầu thất bại (failed request), lập trình viên phải kết nối (SSH) vào từng container, đọc nhật ký và chấp vá thời gian — rất khó với concurrent requests và lệch thời gian (clock skew) giữa containers.



Hình 11.3: Logs phân tán (manual debug) vs Logs tập trung — Search một nơi

11.2.2. Structured Logging — Từ text sang JSON

Bước đầu tiên: chuyển từ **text logs** sang **structured logs** (JSON). Text logs dễ đọc cho người nhưng *rất khó phân tích tự động*.

```
// #sym.times Text log — khó parse, khó filter, khó aggregate
2026-01-15 10:03:02 ERROR [kbm-core] - SQL execution failed for submission 8a3f
user 12ab, Timeout after 30s

// #sym.checkmark Structured log (JSON) — dễ query, filter, aggregate
{"timestamp":"2026-01-15T10:03:02Z","level":"ERROR","service":"kbm-core",
"message":"SQL execution failed","submissionId":"8a3f","userId":"12ab",
"duration_ms":30000,"error":"QueryTimeoutException","traceId":"abc123"}
```

Listing 11.1: Ví dụ Structured Log (JSON)

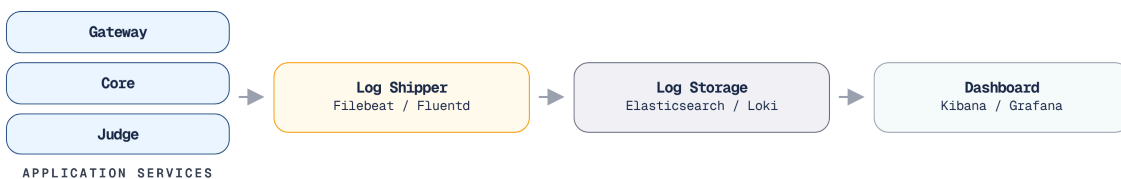
Với structured logs, hệ thống có thể truy vấn (query): “Tìm tất cả các lỗi (errors) trong 24h qua có `error = QueryTimeoutException`” hoặc “Thời gian xử lý trung bình (average) của các lượt nộp bài (submissions) theo giờ” — không thể với text logs.

```
<configuration>
  <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <!-- Tự động trích xuất traceId, spanId từ MDC (nếu dùng Micrometer Tracing) -->
      <includeMdcKeyName>traceId</includeMdcKeyName>
      <includeMdcKeyName>spanId</includeMdcKeyName>
    </encoder>
  </appender>
  <root level="INFO"><appender-ref ref="CONSOLE" /></root>
</configuration>
```

Listing 11.2: Khai báo Logback xuất JSON chứa Trace/Span ID (`logback-spring.xml`)

11.2.3. Centralized Log Aggregation

Sau khi có structured logs, bước tiếp theo là **centralized log aggregation** — gom logs từ tất cả services về một nơi. Newman trong [4a, Ch.8] nhấn mạnh: *centralized logging là yêu cầu bắt buộc* cho microservices, không phải optional.



Hình 11.4: Pipeline gom logs tập trung — từ service đến storage

Để triển khai một pipeline thu thập log tập trung hiệu quả như minh họa ở trên, các tổ chức thường phải đánh đổi giữa tính năng tìm kiếm toàn văn bản (full-text search) chuyên sâu và mức độ tiêu thụ tài nguyên phần cứng. Trong bối cảnh hệ sinh thái cloud-native, có rất nhiều công cụ hỗ trợ nhưng nổi bật nhất vẫn là hai stack công nghệ tiêu chuẩn dưới đây:

Stack	Components	Ưu điểm	Nhược điểm	Phù hợp
ELK	Elasticsearch + Logstash + Kibana	Mature, full-text search mạnh	Resource-heavy (~4GB RAM minimum)	Nhóm quy mô lớn, nhiều logs
PLG	Promtail + Loki + Grafana	Lightweight, tích hợp metrics	Query kém linh hoạt hơn ELK	Nhóm nhỏ-vừa, KBM

Bảng 11.4: So sánh hai stack ELK và PLG

11.2.4. Correlation IDs — Kỹ thuật quan trọng nhất

Dù đã có hệ thống gom log tập trung xuất sắc, việc phân tích nguyên nhân gốc rễ (root cause analysis) vẫn sẽ rất khó khăn nếu các dòng log của cùng một request bị rời rạc. Để giải quyết vấn đề này, Newman trong [4a, Ch.8] mô tả **Correlation ID** (hay Trace ID) là kỹ thuật quan trọng nhất cho debugging microservices. Cơ chế này yêu cầu hệ thống sinh ra một identifier duy nhất gắn vào request từ entry point (Gateway), sau đó truyền qua tất cả các services thông qua HTTP headers và cả Kafka message headers nhằm xâu chuỗi toàn bộ quá trình xử lý.



Hình 11.5: Truyền Correlation ID xuyên suốt request, kể cả qua Kafka

🔔 Phiếu luân chuyển hồ sơ sinh viên

Hãy tưởng tượng một sinh viên nộp hồ sơ xin xét Tốt nghiệp. Hồ sơ này được dán một mã vạch (Correlation ID) duy nhất. Bất kể hồ sơ đi qua Phòng Đào tạo (Core Service), Phòng Tài chính (Finance Service), hay Thư viện (Library Service), mọi phòng ban đều phải quét mã vạch này và ghi vào sổ nhật ký của họ. Nếu hồ sơ bị kẹt, Ban Giám hiệu chỉ cần tìm kiếm mã vạch đó là biết chính xác nó đang nằm ở phòng nào.

Cơ chế hoạt động:

1. **Gateway** generate UUID cho mỗi request incoming, gắn vào HTTP header **X-Correlation-Id**
2. **Downstream services** đọc header, ghi vào **MDC** (Mapped Diagnostic Context) — logging framework tự động gắn kèm vào mọi log entries
3. **Kafka messages** truyền correlation ID qua message headers — nối async flow
4. **Khi debug search** `traceId = abc-123` trong centralized log → thấy toàn bộ vòng đời xử lý request

```

@Component
public class CorrelationIdFilter implements GlobalFilter, Ordered {
    @Override
    public Mono<Void> filter(
        ServerWebExchange exchange, GatewayFilterChain chain) {
        String correlationId = exchange.getRequest().getHeaders()
            .getFirst("X-Correlation-Id");
        if (correlationId == null) {
            correlationId = UUID.randomUUID().toString();
        }

        // 1. Gắn vào HTTP Header để truyền cho downstream service
        ServerHttpRequest mutatedRequest = exchange.getRequest().mutate()
            .header("X-Correlation-Id", correlationId)
            .build();

        // 2. Lưu vào Reactor Context (KHÔNG DÙNG ThreadLocal/MDC ở đây)
        String finalId = correlationId;
        return chain.filter(exchange.mutate().request(mutatedRequest).build())
            .contextWrite(Context.of("CORRELATION_ID", finalId));
    }

    @Override
    public int getOrder() { return -1; }
}

```

Listing 11.3: Tìm Correlation ID an toàn trong Spring Cloud Gateway (WebFlux - Non-blocking)

Every Request Gets a Trace ID

“Ensure that every request entering your system gets a unique identifier, and that this identifier is passed along to all downstream services. Without this, debugging distributed systems is virtually impossible.”

— Sam Newman, *Building Microservices* [4a, Ch.8]

11.3. Chi tiết về Distributed Tracing

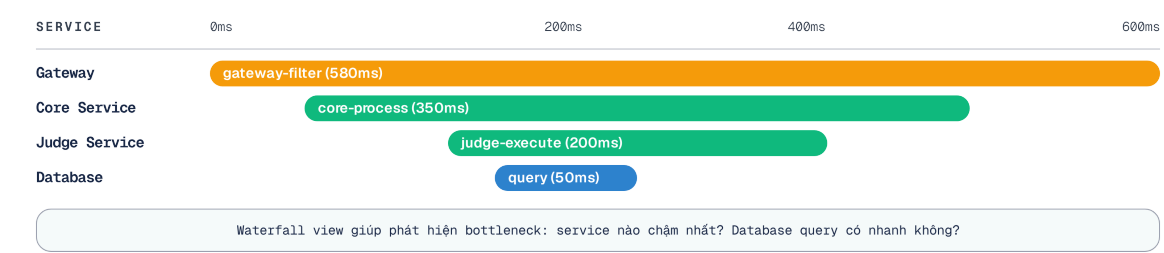
Khi sinh viên phàn nàn hệ thống nộp bài quá chậm, thật khó để biết độ trễ nằm ở đâu nếu không có cơ chế theo dõi dấu vết. Distributed Tracing giải quyết bài toán này bằng cách vẽ lại chính xác đường đi của mỗi yêu cầu.

11.3.1. Vấn đề: request chậm — chậm ở đâu?

Correlation IDs giúp nối logs lại — nhưng không cho biết **mỗi bước mất bao lâu**. Khi sinh viên nộp bài và thời gian phản hồi mất 8 giây (so với mức bình thường là 2 giây), lập trình viên biết yêu cầu đã đi qua Gateway → Core → Judge → MySQL, nhưng không thể xác định 6 giây dư thừa phát sinh ở bước nào nếu không kiểm tra dấu thời gian (timestamps) một cách thủ công.

11.3.2. Distributed Tracing — Tự động đo timing

Distributed tracing ghi lại toàn bộ vòng đời của một request qua tất cả services, bao gồm thời gian mỗi bước (**span**):



Hình 11.6: Gantt chart của một Trace — hiển thị timing cho từng Span

Từ trace này, rõ ràng: **Execute SQL on sandbox** mất 570ms — chiếm 70% tổng thời gian. Đây là nút thắt cổ chai (bottleneck) cần được tối ưu hóa. Không có tracing, lập trình viên buộc phải phỏng đoán, hoặc tra cứu thủ công từng dấu thời gian trong nhật ký.

11.3.3. Concepts: Trace, Span, Context Propagation

🔗 Quy trình Nhập học

- Trace** – Là tổng thời gian từ lúc tân sinh viên bước vào trường đến lúc cầm thẻ sinh viên ra về (ví dụ: 120 phút).
- Span** – Là thời gian sinh viên dừng lại ở từng trạm. Trạm “Đóng học phí” là một **Parent Span** (30 phút).
- Child Span** – Bên trong trạm Đóng học phí có 2 khâu nhỏ: “Xếp hàng chờ” (25 phút) và “Quẹt thẻ ngân hàng” (5 phút). Nhờ chia nhỏ thành child span, chúng ta biết ngay “cổ chai” (bottleneck) không phải do hệ thống ngân hàng chậm, mà do thời gian chờ đợi xếp hàng quá lâu.

Ảnh dụ trên giúp hình dung trực quan quy trình tracing. Bảng dưới đây hệ thống hóa bốn khái niệm cốt lõi — Trace, Span, Parent-child, và Context propagation — cùng ví dụ cụ thể trong flow submit bài của KBM.

Concept	Mô tả	Ví dụ
Trace	Toàn bộ vòng đời xử lý của một request	Submit bài từ browser → score trả về
Span	Một bước trong trace (một unit of work)	“Core Service: save submission” (40ms)
Parent-child	Span có thể chứa child spans	“Judge Service” span chứa “Execute SQL” và “Compare”
Context propagation	Truyền trace context qua service boundaries	HTTP header <code>traceparent</code> , Kafka message header

Bảng 11.5: Các khái niệm cơ bản trong Distributed Tracing

11.3.4. Sampling Strategies

Trong môi trường thực tế (production), việc truy vết **100% requests** sẽ tiêu tốn rất nhiều dung lượng lưu trữ và năng lực xử lý. Do đó, cần có chiến lược lấy mẫu (sampling strategy):

Strategy	Mô tả	Khi nào dùng
Head-based	Quyết định sample tại entry point (Gateway)	Đơn giản, dễ implement
Tail-based	Quyết định sau khi trace hoàn thành (giữ traces lỗi, bỏ traces thành công)	Capture 100% errors, giảm storage
Rate-based	Sample N% requests	Production high-traffic (1-10%)
Adaptive	Tự động điều chỉnh sample rate theo traffic volume	Enterprise, dynamic workloads

Bảng 11.6: Sampling strategies cho Tracing

Với LMS: contest mode (100+ concurrent users) nên dùng rate-based 10-20%; ngoài contest dùng 100%.

11.3.5. Tracing trong Spring Boot 3.x

Spring Boot 3.x tích hợp sẵn **Micrometer Tracing** — abstraction layer cho distributed tracing. **OpenTelemetry (OTel)** hiện nay đã trở thành chuẩn mực công nghiệp thống nhất cho cả 3 trụ cột (Logs, Metrics, Traces), định nghĩa chuẩn truyền tải OTLP.

Để bật tracing và xuất dữ liệu ra OTel Collector, chúng ta chỉ cần khai báo dependencies và cấu hình trong `application.yml` mà không phải sửa code logic:

```
<!-- Spring Boot 3 Actuator -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<!-- Micrometer OTel Bridge -->
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-tracing-bridge-otel</artifactId>
</dependency>
<!-- OTLP Exporter -->
<dependency>
  <groupId>io.opentelemetry</groupId>
  <artifactId>opentelemetry-exporter-otlp</artifactId>
</dependency>
```

Listing 11.4: Khai báo dependencies cho Tracing (`pom.xml`)

Tuy nhiên, Distributed Tracing chỉ phát huy tối đa hiệu quả khi được kết hợp với Logs. Tracing giúp định vị ‘nút thắt’ (bottleneck), còn Logs giải thích nguyên nhân gốc rễ (root cause).

Tracing Complements Logging

“Tracing xác định đường đi và thời gian xử lý của request, trong khi Logging giải thích nguyên nhân gây ra vấn đề. Sử dụng trace để nhận diện điểm nghẽn (bottleneck) và dùng logs để hiểu nguyên nhân gốc rễ (root cause). Hai công cụ này bổ trợ cho nhau và không thể thay thế lẫn nhau.”

— Tổng hợp từ Richardson [2a, Ch.11] và Newman [4a, Ch.8]

```

management:
  endpoints:
    web:
      exposure:
        include: health,prometheus # Expose metrics
  tracing:
    sampling:
      probability: 1.0 # Sample 100% (cho môi trường dev/staging)
  otlp:
    tracing:
      endpoint: http://otel-collector:4318/v1/traces

```

Listing 11.5: Cấu hình OTLP Exporter và Sampling (`application.yml`)

Với cấu hình này, Spring Boot **tự động** truyền trace context qua HTTP headers theo chuẩn **W3C Trace Context** (header `traceparent`, thay cho chuẩn B3 cũ). Để truyền qua Kafka, cần bật observation ở *cả hai phía*: `spring.kafka.template.observation-enabled=true` cho producer (`KafkaTemplate`) và `spring.kafka.listener.observation-enabled=true` cho consumer — thiếu phía producer thì trace sẽ đứt đoạn tại broker. Không cần viết code tracing thủ công cho phần lớn use cases.

11.3.6. Kiến trúc triển khai Observability Pipeline

Khi hệ thống lớn lên, ứng dụng không nên gửi trực tiếp data đến Jaeger/Prometheus. Có 3 mô hình triển khai:

1. **Direct SDK**: Ứng dụng gửi trực tiếp dữ liệu đến hệ thống lưu trữ (Độ liên kết cao - Tight coupling, dễ gây nghẽn mạng).
2. **Local Agent**: Ứng dụng gửi dữ liệu nội bộ tới Tác nhân (Agent) (DaemonSet). Tác nhân tự quản lý bộ đệm (buffer) và gộp lô (batching).
3. **Centralized Collector**: Tất cả Agent gửi về một cụm Gateway Collector trung gian để mask PII, filter và routing trước khi lưu trữ.

⚠ Nút thắt OTel Collector

Trong mô hình Centralized Collector, cụm Collector là “Single Point of Processing”.

OOM (Out of Memory) – Rất dễ xảy ra nếu dùng **Tail-based sampling**, vì Collector phải lưu trữ tạm toàn bộ Span của mọi request trên RAM để đợi quyết định lưu trữ hay loại bỏ ở bước cuối cùng.

CPU Bound & Network Saturation – Do xử lý parse log/traces và lượng data khổng lồ.

Mitigation Strategies: Triển khai kiến trúc Tiered (Agent ở biên, Gateway ở trung tâm), dùng Apache Kafka ở giữa làm buffer hấp thụ đợt burst traffic, và cài đặt **Log Retention Policy** (chỉ lưu hot logs trong 7-14 ngày trên Loki, rồi archive ra S3 để tối ưu chi phí lưu trữ).

11.4. Metrics, SLI/SLO, và Alerting Strategy

Hệ thống cần các chỉ tiêu chất lượng rõ ràng để cảnh báo đội ngũ kỹ thuật kịp thời — trước khi sự cố kịp làm gián đoạn việc học của sinh viên.

11.4.1. Metrics — Đo lường hành vi theo thời gian

Trước khi thiết lập cảnh báo dựa trên metrics, cần tránh một lỗi thiết kế phổ biến trong thu thập dữ liệu chuỗi thời gian:

⚠ Cardinality Explosion

Một sai lầm kinh điển của các nhóm mới làm quen với microservices là đưa `userId` hoặc `submissionId` vào làm “label” (tag) cho Prometheus metrics. Metrics được thiết kế cho **Low Cardinality** (số lượng tag hữu hạn, vd: `status=200`, `method=GET`). Nếu nhóm phát triển gắn `userId` làm nhãn, cơ sở dữ liệu chuỗi thời gian (TSDB) sẽ tạo ra hàng triệu time-series khác nhau và gây quá tải, dẫn đến sự sụp đổ (crash) của hệ thống ngay lập tức. Để tra cứu theo `userId`, hãy dùng Logs hoặc Traces.

Logs ghi lại *sự kiện đã xảy ra*. Metrics đo lường *xu hướng theo thời gian* — quan trọng cho **early detection** phát hiện vấn đề trước khi người dùng báo lỗi.

Richardson trong [2a, Ch.11] mô tả bốn observability pattern cần triển khai:

Pattern	Mô tả	Ví dụ KBM
Health Check API	Endpoint <code>/health</code> cho hệ thống điều phối (orchestrator) biết dịch vụ đang hoạt động hay đã ngừng (up/down)	Eureka lưu và công bố trạng thái từ health check; LoadBalancer dựa vào đó để chọn instance
Application metrics	Business-level metrics	Submission rate, judge duration, error rate
Infrastructure metrics	System-level metrics	CPU, memory, DB connections, Kafka consumer lag
Audit logging	Ghi lại hành vi user cho compliance	User login, role changes, data access

Bảng 11.7: Bốn loại metrics cần thu thập

11.4.2. Bốn loại Metrics cơ bản của Prometheus

Trước khi áp dụng RED/USE, lập trình viên cần hiểu 4 kiểu dữ liệu đo lường:

1. **Counter**: Bộ đếm chỉ tăng lên (vd: Tổng số requests). Tốt để tính Rate.
2. **Gauge**: Giá trị có thể tăng/giảm (vd: CPU usage, Active Connections). Tốt để tính Saturation.
3. **Histogram**: Nhóm dữ liệu vào các “bucket” (vd: Số request xong trong $<100\text{ms}$, $<500\text{ms}$). Nên dùng cho Latency Percentile (P99) vì phía server tính lại và *gộp* được số liệu qua nhiều instance.
4. **Summary**: Tính sẵn quantile ở phía client — chính xác cho từng instance nhưng không gộp được giữa các replica, nên ít phù hợp với hệ phân tán nhiều bản sao.

💡 Exemplars: Cầu nối Metrics & Traces

Công nghệ hiện đại cho phép nhúng trực tiếp một `traceId` đại diện vào trong các bucket của Prometheus Histogram (gọi là **Exemplars**). Khi quan sát thấy một đỉnh trễ (latency spike) bất thường trên Grafana, kỹ sư có thể nhấp trực tiếp vào điểm dữ liệu đó để chuyển hướng đến Jaeger và phân tích chính xác trace gây ra sự chậm trễ.

Để quy chuẩn hóa việc đo lường trạng thái hệ thống, ngành công nghiệp thường sử dụng hai khuôn mẫu tiêu chuẩn: RED (cho dịch vụ) và USE (cho tài nguyên).

11.4.3. Hai framework đo lường: RED và USE

RED (Dùng cho Services - request-facing) – Bao gồm **R**ate, **E**rrors, và **D**uration. Ví dụ: Submission rate là 50 req/min, Error rate là 2%, P99 latency là 3.2s.

USE (Dùng cho Resources - infrastructure) – Bao gồm **U**tilization, **S**aturation, và **E**rrors. Ví dụ: CPU là 45%, DB connection pool là 8/10 active, Disk I/O errors là 0.

Kết hợp RED (cho services) + USE (cho resources) để giám sát cả hai khía cạnh của hệ thống. Metrics phát hiện “submission latency tăng” (RED), investigation cho thấy “DB connection pool saturated” (USE) → root cause: connection leak.

11.4.4. SLI/SLO/SLA – Từ metrics đến cam kết chất lượng

Metrics chỉ hữu ích khi biết **ngưỡng nào là chấp nhận được** Framework SLI/SLO/SLA cung cấp ngôn ngữ chung:

📖 Thư viện trường học

SLI (Chỉ số thực tế) – Thời gian thực tế thủ thư dùng để xếp một cuốn sách trả lại lên kệ (vd: 1.5 giờ).

SLO (Mục tiêu nội bộ) – KPI mà Giám đốc Thư viện đặt ra cho nhân viên: “99% sách trả lại phải được xếp lên kệ trong vòng 2 giờ”.

SLA (Cam kết đền bù) – Cam kết trong Sổ tay Sinh viên: “Nếu sách người mượn trả chưa được hệ thống cập nhật trong ngày, nhà trường cam kết sẽ miễn trừ mọi khoản phạt trễ hạn tự động phát sinh”.

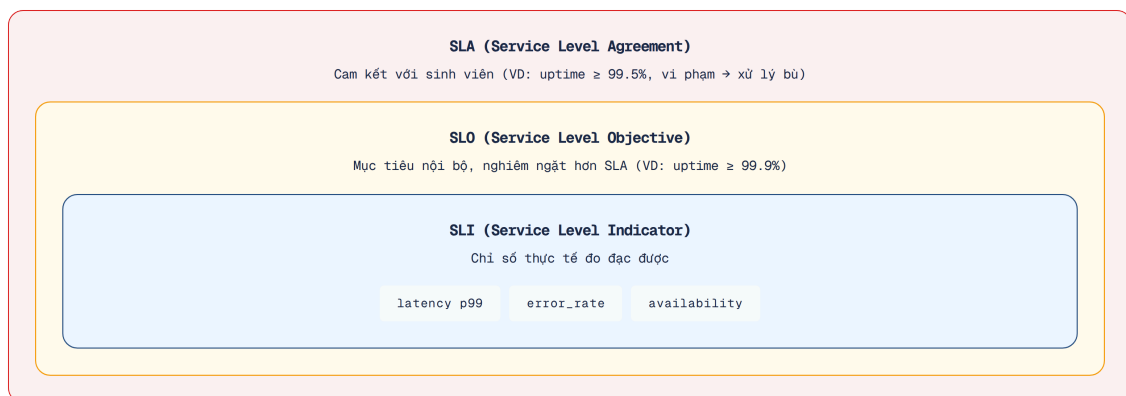
Để đảm bảo chất lượng vận hành, hệ sinh thái đo lường được chia thành ba khái niệm chính:

SLI (Service Level Indicator) – Là metric cụ thể được hệ thống đo lường trực tiếp để đánh giá chất lượng dịch vụ trong thực tế. Trong KBM, ví dụ tiêu biểu là đo “Latency P99 của submission flow”.

SLO (Service Level Objective) – Là mục tiêu nội bộ mà đội ngũ kỹ thuật tự đặt ra đối với một SLI nhất định. Ví dụ: “99% submissions phải trả kết quả trong vòng chưa tới 5 giây”.

SLA (Service Level Agreement) – Là những cam kết chính thức với khách hàng, có đi kèm với các hình phạt tài chính hoặc hậu quả pháp lý rõ ràng nếu hệ thống bị vi phạm. Ví dụ: “Hệ thống cam kết availability đạt mức 99.5% tổng thời gian trong một học kỳ”.

Các khái niệm trên phân biệt rõ ba mức độ hay bị nhầm lẫn: SLI là con số đo được, SLO là mục tiêu nội bộ mà nhóm phát triển tự đặt, SLA là cam kết có ràng buộc pháp lý với bên ngoài. Hình dưới minh họa mối quan hệ lồng nhau giữa chúng — SLI nằm bên trong SLO, SLO nằm bên trong SLA.



Hình 11.7: Mối quan hệ giữa SLI, SLO và SLA

Với KBM — hệ thống giáo dục nội bộ phục vụ sinh viên của trường — nhóm phát triển không cần SLA chính thức vì không có hợp đồng thương mại. Tuy nhiên, SLO nội bộ vẫn rất quan trọng: chúng đặt ngưỡng để nhóm phát triển biết khi nào cần ưu tiên khắc phục lỗi, khi nào chấp nhận được. Bảng dưới đề xuất bốn SLO phù hợp với bối cảnh KBM:

SLI	SLO	Lý giải
Submission latency P99	< 5 giây	Sinh viên phải chờ đợi lâu dẫn đến tâm lý thất vọng và ngừng sử dụng hệ thống
Availability trong contest	99.9% (~8.7h downtime/năm)	Contest có thời gian giới hạn — downtime ảnh hưởng trực tiếp điểm
Grading accuracy	100%	Việc sai sót sẽ ảnh hưởng trực tiếp đến kết quả học thuật; do đó, độ chính xác phải được đảm bảo tuyệt đối
API error rate	< 1%	Tần suất lỗi cao sẽ làm giảm mức độ tin cậy của người dùng đối với hệ thống

Bảng 11.8: SLO đề xuất cho KBM

11.4.5. Alerting Strategy

Tuy nhiên, nếu đội ngũ kỹ sư cấu hình cảnh báo quá nhạy cảm hoặc sai cách, họ sẽ nhanh chóng rơi vào trạng thái “mệt mỏi vì cảnh báo” (**alert fatigue**). Hiện tượng này xảy ra khi có quá nhiều cảnh báo giả (false positives) liên tục phát sinh, khiến các kỹ sư on-call dần sinh ra tâm lý chủ quan và bỏ qua ngay cả những sự cố thực sự nghiêm trọng. Để chống lại **alert fatigue**, một cơ chế quan trọng cần áp dụng là **Alert grouping** (gom nhóm cảnh báo) — thay vì gửi 100 thông báo riêng lẻ khi một dịch vụ ngừng hoạt động (down) kéo theo hàng loạt lỗi liên quan, hệ thống sẽ gom chúng lại thành một thông báo duy nhất theo nguyên nhân gốc rễ.

Level	Khi nào	Kênh	Hành động	Ví dụ
Critical	SLO bị vi phạm, ảnh hưởng đến người dùng ngay lập tức	PagerDuty, phone call	Kỹ sư trực hệ thống (on-call) phải xử lý ngay	Dịch vụ Judge ngừng hoạt động, nộp bài thất bại
Warning	Gần vi phạm SLO, cần chú ý	Slack, email	Xem xét trong giờ làm việc	Latency P99 đạt 80% threshold
Info	Anomaly nhưng không ảnh hưởng người dùng	Dashboard only	Xem xét sau hoặc phân tích định kỳ	Kafka consumer lag tăng nhẹ

Bảng 11.9: Chiến lược Alerting theo mức độ nghiêm trọng

Ba tầng cảnh báo ở trên giúp tránh tình trạng “mọi sự cố đều được coi là critical”. Khi áp dụng thực tiễn, việc xây dựng tiêu chí cảnh báo cần gắn kết trực tiếp với trải nghiệm người dùng (thông qua SLO), thay vì dựa trên chỉ số kỹ thuật thuần túy. Chẳng hạn, nếu CPU tăng cao nhưng độ trễ vẫn trong giới hạn và người dùng không bị ảnh hưởng, thì không cần đánh thức kỹ sư on-call vào lúc 3 giờ sáng.

Alert on Symptoms, Not Causes

“Alert khi SLO có nguy cơ bị vi phạm, không phải khi một metric thay đổi. CPU 90% nhưng latency bình thường? Không cần alert. CPU 50% nhưng latency vượt SLO? Alert ngay.”

— Nguyên tắc SRE, dựa trên Google SRE book

11.5. Error Handling Strategy — Nhất quán xuyên Services

Một hệ thống nhiều thành phần thường gặp tình trạng mỗi dịch vụ trả về một định dạng báo lỗi khác nhau. Để tránh gây nhầm lẫn cho ứng dụng frontend và sinh viên, hệ thống cần chuẩn hóa cách xử lý lỗi.

Vấn đề: mỗi service trả error format khác nhau. Khi hệ thống bao gồm nhiều dịch vụ do các nhóm khác nhau phát triển, định dạng phản hồi lỗi (error responses) rất dễ mất đi tính nhất quán (*inconsistent*). Dịch vụ A trả về lỗi trong trường `description`, Dịch vụ B lại sử dụng trường `message`, Gateway trả format hoàn toàn khác. Ứng dụng khách (client/frontend) phải xử lý nhiều format — mã nguồn phình phồng phức tạp và trải nghiệm người dùng (UX) kém đi. Do đó cần thiết lập nguyên tắc chuẩn hóa.

11.5.1. Nguyên tắc thiết kế Error Handling

Richardson trong [2a, Ch.11] và Newman trong [4a, Ch.8] đề xuất các nguyên tắc cho error handling trong microservices:

1. Centralized exception handler

— Mỗi service có một handler tập trung (trong Spring Boot: `@ControllerAdvice`) xử lý tất cả exceptions → chuyển thành format chuẩn. Đặt trong shared library để mọi service dùng chung.

2. Error code taxonomy

— Phân loại errors theo namespace để client và monitoring system hiểu nhanh:

Range	Category	Ví dụ
1xxx	Authentication & Authorization	1001: Unauthorized, 1002: Token expired
2xxx	Resource not found	2001: Question not found, 2002: Contest not found
3xxx	Business logic violation	3001: Contest chưa bắt đầu, 3002: Hết lượt nộp
4xxx	Infrastructure / dependency	4001: Judge service unavailable, 4002: Kafka send failed
9xxx	Internal errors	9999: Unexpected error

Bảng 11.10: Taxonomy hệ thống mã lỗi (Error Code)

3. Unified response format

— Tất cả error responses dùng cùng cấu trúc:

```
// #sym.checkmark Format nhất quán — client chỉ cần parse một structure
{
  "errorCode": 4001,
  "description": "Judge service unavailable",
  "timestamp": "2026-01-15T10:03:02Z",
  "traceId": "abc-123"
}
```

Listing 11.6: Unified Error Response payload

4. Never leak internals

— Stack traces, SQL queries, internal paths không bao giờ xuất hiện trong error response. Log chi tiết ở server side (với traceId để correlation), trả thông báo generic cho client.

Mối liên hệ giữa Error Handling và Observability. Trong thực tế, các mã lỗi tập trung không chỉ phục vụ client — chúng là **nguồn dữ liệu quan trọng cho metrics và alerting**:

- Count errors by code → phát hiện patterns: `JUDGE_UNAVAILABLE` tăng đột biến → alert
- Group errors by category → dashboard: authentication issues vs business logic vs infrastructure
- Error rate per service → SLI cho reliability

❗ Error format inconsistency trong KBM

KBM sử dụng `@ControllerAdvice` với `ErrorCode` enum trong shared library — đúng pattern. Tuy nhiên, **Gateway** (Spring Cloud Gateway dùng WebFlux) xử lý JWT errors *bên ngoài* `@ControllerAdvice` (WebMVC pattern). Kết quả: Gateway trả error dạng `{"message": "..."}` trong khi các services trả `{"description": "..."}` — field name khác nhau. Ứng dụng giao diện (frontend) phải xử lý đồng thời cả hai định dạng này.

Migration path (1) Tạo `GatewayExceptionHandler` riêng cho WebFlux, chuẩn hóa response format, (2) Đảm bảo tất cả error responses dùng cùng field names, (3) Thêm `traceId` vào error response để client có thể báo cáo cho đội ngũ hỗ trợ.

11.6. Health Checks và Service Discovery

Service Discovery và API Gateway chỉ có thể tự động ngắt định tuyến khỏi dịch vụ gặp sự cố khi có căn cứ: đó là vai trò của Health Checks (kiểm tra sức khỏe).

11.6.1. Health Checks — Nền tảng cho tự động hóa

Richardson trong [2a, Ch.11] mô tả **Health Check API** là observability pattern cơ bản nhất: mỗi dịch vụ cần cung cấp (expose) một endpoint `/health` để các công cụ điều phối (orchestration tools) nhận biết trạng thái hoạt động của nó.

Health checks phân loại theo mục đích:

Loại	Câu hỏi trả lời	Ý nghĩa cho orchestrator	Ví dụ KBM
Liveness	“Process còn chạy?”	Restart container nếu tiến trình không phản hồi	JVM alive, không deadlock
Readiness	“Sẵn sàng nhận traffic?”	Ngừng route traffic nếu dịch vụ chưa sẵn sàng	Database connected, Kafka connected
Startup	“Đã khởi tạo xong?”	Chờ quá trình khởi động hoàn tất mới check liveness	Schema migration done, cache warmed

Bảng 11.11: Ba loại Health Checks

Trong KBM, health checks tham gia vào định tuyến theo một chuỗi trách nhiệm rõ ràng: instance Core Service báo **DOWN** qua endpoint health → **Eureka** (Service Registry) cập nhật trạng thái trong danh bạ → Spring Cloud LoadBalancer ở Gateway loại instance đó khỏi danh sách lựa chọn và chuyển traffic sang instance khỏe mạnh. DevOps Lab bổ sung health/readiness ở runtime layer để router chỉ chuyển traffic đến workspace sẵn sàng. Còn việc *khởi động lại* instance hòng thuộc về tầng orchestrator/supervisor (Docker restart policy, Kubernetes) — kết hợp cả hai tầng mới tạo thành bức tranh **self-healing** hoàn chỉnh không cần can thiệp thủ công.

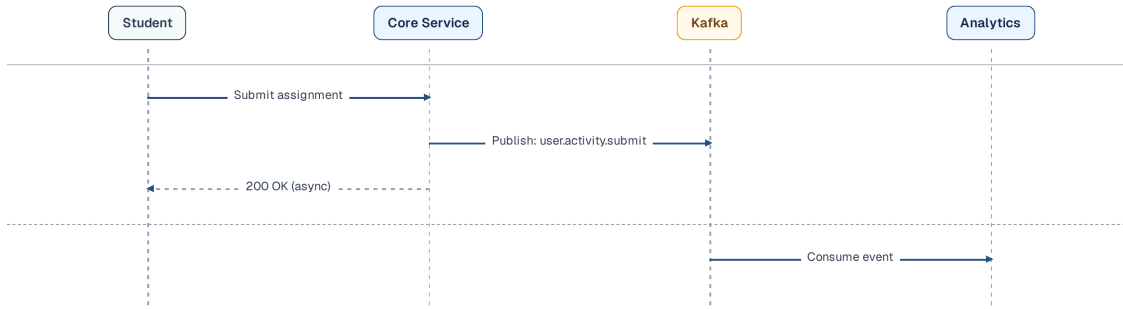
⚠ Eureka không tự suy ra trạng thái health của service

Một cấu hình dễ bỏ sót: mặc định Eureka client chỉ gửi *heartbeat* để báo tiến trình còn sống, nên một service vẫn được công bố **UP** dù endpoint health đã **DOWN** (mất kết nối database chẳng hạn). Traffic khi đó tiếp tục được chuyển đến một instance không phục vụ được. KBM bật `eureka.client.healthcheck.enabled=true` để trạng thái từ Spring Boot Actuator được truyền lên registry. Khi bật, cần cân nhắc chính health model của hệ thống: nếu endpoint health kiểm tra quá sâu vào các phụ thuộc bên ngoài, một sự cố nhất thời có thể khiến hàng loạt instance bị rút khỏi danh bạ cùng lúc (xem thêm phần phân biệt liveness và readiness ở cuối chương).

11.7. User Activity Tracking — Observability ở mức Business

Ba trụ cột (logs, metrics, traces) giúp quan sát **hệ thống kỹ thuật**. Nhưng với KBM — một LMS — cần thêm: quan sát **hành vi học tập** (learning analytics). Sinh viên đang học gì? Bài nào khó nhất? Khi nào sinh viên hoạt động nhiều nhất?

KBM đã triển khai **User Activity Tracking** qua Kafka pipeline: khi sinh viên xem câu hỏi, nộp bài, hoặc tham gia contest, Core Service sẽ phát (publish) các sự kiện theo dõi lên Kafka topic. Consumer sẽ tiếp nhận và ghi dữ liệu — quá trình này diễn ra hoàn toàn bất đồng bộ (async) và không chặn (non-blocking), do đó không làm ảnh hưởng đến thời gian phản hồi (response time) của ứng dụng.



Hình 11.8: User Activity Tracking pipeline qua Kafka

Data	Câu hỏi trả lời được	Ứng dụng
View patterns	“Câu hỏi nào được xem nhiều nhất?”	Đề xuất bài tập phù hợp
Submit patterns	“Sinh viên nào nộp bài nhiều lần?”	Phát hiện sinh viên cần hỗ trợ
Time patterns	“Peak hours là khi nào?”	Scale infrastructure phù hợp
Contest behavior	“Bao nhiêu % sinh viên hoàn thành contest?”	Đánh giá hiệu quả giảng dạy

Bảng 11.12: Ứng dụng của User Activity Tracking

Đây là điểm giao giữa technical observability và **domain-specific analytics** — một giá trị riêng mà observability mang lại cho LMS, vượt xa việc chỉ phục vụ đội ngũ vận hành.

11.8. Chaos Engineering — Kiểm tra Resilience chủ động

Observability cho phép **phát hiện** sự cố. Nhưng làm sao biết hệ thống sẽ phản ứng thế nào *trước khi* sự cố thật xảy ra? **Chaos Engineering** — phương pháp được Netflix phổ biến qua Chaos Monkey (2010) — trả lời câu hỏi này bằng cách **chủ động inject failures** vào production/staging và quan sát phản ứng. Cần lưu ý rằng Chaos Engineering là định hướng ở giai đoạn trưởng thành cao (từ Level 4 trở lên), hệ thống KBM trước mắt chỉ dùng giả lập lỗi thủ công (manual error simulation) để kiểm chứng tính bền bỉ.

Nguyên tắc (theo *Principles of Chaos Engineering* — principlesofchaos.org):

1. **Xác định “steady state”** — hệ thống hoạt động bình thường trông như thế nào? (metrics: latency < 200ms, error rate < 0.1%, throughput > 100 req/s)
2. **Đặt giả thuyết** — “Nếu Judge Service ngừng hoạt động (down), submissions vẫn được queue và xử lý khi Judge restart”
3. **Inject failure** — Kill Judge Service container
4. **Observe** — Steady state có bị phá vỡ không? System recover trong bao lâu?
5. **Khắc phục điểm yếu** — Nếu trạng thái ổn định bị phá vỡ, cần sửa lỗi trên hệ thống (fix system) thay vì điều chỉnh lại kịch bản kiểm thử (fix test)

💡 Diễn tập phòng cháy chữa cháy

Chaos Engineering giống như nhà trường cố tình bấm còi báo “Diễn tập phòng cháy chữa cháy” đột xuất ở Ký túc xá lúc 9 giờ tối. Mục đích là chủ động xem sinh viên có thoát hiểm kịp trong 5 phút không, của thoát hiểm có mở không — thay vì đợi đến khi có hỏa hoạn thật mới phát hiện ra cửa đã bị rỉ sét không mở được.

Tương tự như việc ban quản lý ký túc xá chuẩn bị nhiều kịch bản diễn tập khác nhau (chập điện, rò rỉ nước, hỏa hoạn), kỹ sư hệ thống cũng có các kịch bản tiêm lỗi đa dạng để kiểm thử độ bền bỉ của nền tảng:

Loại failure injection	Ví dụ	Tool
Process failure	Kill container/service	Chaos Monkey, <code>docker stop</code>
Network failure	Delay, packet loss, partition	Toxiproxy, Gremlin
Resource exhaustion	CPU 100%, disk full, memory leak	stress-ng, Gremlin
Dependency failure	Database down, Kafka unavailable	Litmus, manual

Bảng 11.13: Phân loại Failure Injection trong Chaos Engineering

Bốn loại failure injection trên phủ gần hết failure domain trong hệ thống phân tán. Tuy nhiên, nhóm phát triển không cần chạy tất cả cùng lúc — nên bắt đầu từ ba thí nghiệm đơn giản nhất, vừa tầm với hạ tầng Docker Compose hiện tại của KBM, và đã đủ kiểm chứng resilience patterns đã thiết kế ở các chương trước: Các kịch bản Chaos experiments tiêu biểu dành cho nền tảng KBM bao gồm:

Kill Judge Service – Giả thuyết đặt ra là các submissions của sinh viên sẽ tự động được queue an toàn trong Kafka và xử lý tiếp khi service restart. Kết quả mong đợi là Kafka retention sẽ giữ toàn bộ messages và Judge bắt kịp tiến độ (catch up) ngay khi dịch vụ được phục hồi (restart).

Network delay 500ms giữa Gateway↔Core – Giả thuyết cho rằng sinh viên sẽ trải nghiệm tốc độ phản hồi chậm hơn đôi chút nhưng hệ thống hoàn toàn không trả về lỗi. Kết quả mong đợi là cấu hình Circuit breaker và timeout sẽ hoạt động chính xác để ngăn chặn tình trạng cascade failure.

PostgreSQL restart – Giả thuyết là Core Service có khả năng tự động reconnect lại cơ sở dữ liệu. Kết quả mong đợi là Connection pool (HikariCP) tự động xử lý toàn trình việc kết nối lại mà không yêu cầu khởi động lại ứng dụng.

Điều quan trọng là bắt đầu — không cần tool phức tạp. Docker CLI đã là đủ cho chaos experiments đầu tiên, và giá trị mang lại là sự tự tin rằng hệ thống thực sự recover được.

💡 Chaos Engineering từ Staging

Không cần Chaos Monkey hoặc tools phức tạp. Bắt đầu đơn giản: `docker stop kbm-judge-sql` trên staging → quan sát logs → xem system recover không. Đây đã là chaos engineering. Khi nhóm đã quen, nâng lên: automated chaos experiments chạy trong CI/CD pipeline.

11.8.1. Industry Case Study: Netflix Simian Army

Netflix là pioneer của Chaos Engineering. Năm 2010, sau khi migrate từ datacenter sang AWS, Netflix tạo **Chaos Monkey** — tool tự động kill random EC2 instances trong production. Mục đích: nếu system survive random failures hàng ngày, nó sẽ survive failures thật.

Chaos Monkey phát triển thành **Simian Army** — tập hợp tools chuyên biệt:

Tool	Chức năng	Bài học
Chaos Monkey	Kill ngẫu nhiên các instance	Services phải stateless, tự khởi động lại được
Chaos Kong	Giả lập sự cố toàn bộ region	Multi-region failover phải hoạt động
Latency Monkey	Tiêm độ trễ mạng (network delay)	Timeout + circuit breaker phải được cấu hình đúng
Conformity Monkey	Kiểm tra instance tuân thủ best practices	Tự động hóa thay cho kiểm tra thủ công

Bảng 11.14: Netflix Simian Army

Kết quả: Netflix duy trì độ sẵn sàng cao (mục tiêu 99.99%) khi phục vụ hơn 200 triệu subscribers. Casey Rosenthal — người dẫn dắt nhóm Chaos Engineering của Netflix, sau này đồng sáng lập Verica — cùng Nora Jones là đồng tác giả *Chaos Engineering: System Resiliency in Practice* (O'Reilly, 2020).

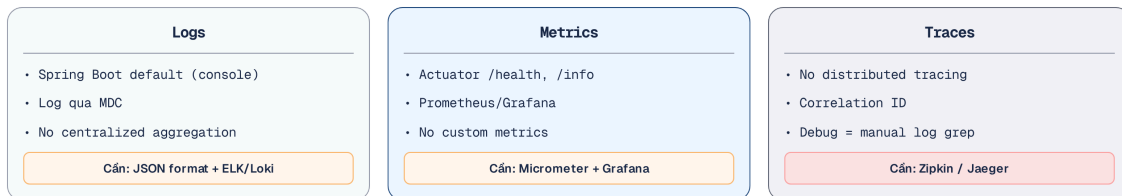
Bài học cho hệ thống nhỏ: LMS không cần Simian Army. Nhưng nguyên tắc cốt lõi áp dụng: *nếu đội ngũ phát triển e ngại việc vô hiệu hóa đột ngột (kill) một dịch vụ, đó là dấu hiệu hệ thống thiếu resilience.* Bắt đầu manual (`docker stop`), tiến tới automated (CI/CD chaos stage).

Nguồn: Netflix Technology Blog, “The Netflix Simian Army,” 2011 (netflixtechblog.com). Casey Rosenthal et al., *Chaos Engineering: System Resiliency in Practice*, O'Reilly, 2020.

11.9. Case Study: KBM

Phần này phân tích hiện trạng giám sát của KBM và vạch lộ trình nâng cấp tương ứng.

11.9.1. Hiện trạng — Đánh giá theo Maturity Model



Hình 11.9: Đánh giá hiện trạng Observability của KBM

Đánh giá tổng thể: Level 2 đang tiến lên Level 3 — KBM đã có Actuator/health ở một số services và nền tảng Prometheus/Grafana cho một phần hệ thống, nhưng vẫn cần chuẩn hóa ID tương quan (correlation ID), quy trình thu thập log (log aggregation) và truy vết (tracing) xuyên suốt ranh giới các dịch vụ (boundaries). Khi yêu cầu nộp bài phản hồi chậm hoặc thất bại, lập trình viên không nên phải mở nhật ký của từng container và ghép dấu thời gian (timestamps) thủ công.

Component	Hiện trạng	Maturity Level
Logging	Java services có log qua MDC ở một số nơi; DevOps Lab dùng zerolog; cần gom tập trung	🟡 Level 2
Metrics	Actuator/health + Prometheus/Grafana ở một phần hệ thống	🟡 Level 2
Tracing	Correlation ID có nền tảng nhưng chưa chuẩn hóa end-to-end	🟡 Level 2
Error handling	@ControllerAdvice + ErrorCode tốt, Gateway inconsistent	🟢 Level 3
User tracking	Kafka-based async — tốt	🟢 Level 3

Bảng 11.15: Mức độ trưởng thành observability phân theo component

11.9.2. Phân tích theo business context

KBM phục vụ sinh viên học SQL, mạng, DevOps và các hoạt động LMS. Điểm đặc biệt là hệ thống có nhiều mode hoạt động khác nhau — mỗi mode có traffic pattern, yêu cầu latency, và mức chịu lỗi riêng. Observability strategy không thể áp dụng một size cho tất cả: contest mode cần alert real-time, practice mode chỉ cần basic monitoring. Bảng dưới phân tích chi tiết từng mode:

Mode	Đặc điểm	SLO cần thiết	Observability cần
Practice mode	Sinh viên tự luyện tập, traffic thấp, không deadline	Latency < 10s OK, downtime chấp nhận được	Basic monitoring đủ
Contest mode	100+ sinh viên submit đồng thời, có deadline, leaderboard real-time	Latency < 5s, availability 99.9%	Metrics + alerts bắt buộc
DevOps runtime	Lab Workspace/container/network runtime biến động	Readiness chính xác, isolation lỗi	Promtail/Loki/Grafana + runtime metrics

Bảng 11.16: Các chế độ hoạt động chính của LMS

Trong contest mode, **không có observability = thiếu tầm nhìn hệ thống (flying blind)**

- Judge Service overloaded → submissions queue up → leaderboard outdated → sinh viên phàn nàn
- Không có custom metrics → không biết judge duration tăng cho đến khi student report
- Không có tracing → không biết bottleneck ở Judge hay ở Kafka consumer

📌 Observability chưa chuẩn hóa end-to-end

KBM đã có một số mảnh observability đúng hướng: Actuator/Prometheus cho Java services, X-Request-ID /MDC ở shared layer, zerolog ở Go services, và PLG stack (Promtail/Loki/Grafana) cho DevOps Lab. Tuy nhiên, hệ thống vẫn còn thiếu sự đồng nhất trên toàn chuỗi giao dịch.

Khoảng trống lớn nhất hiện tại là **chuẩn hóa end-to-end**: cùng một request hoặc batch cần đi qua Gateway → service → Kafka/DB queue → Judge/runtime mà vẫn giữ được trace/correlation ID nguyên

ven. Việc thiếu hụt này khiến quá trình gỡ rối trở nên gián đoạn và khó theo dõi toàn bộ luồng xử lý thực tế.

Để khắc phục, chúng ta cần một **Migration path** (incremental — mỗi phase đều mang lại giá trị ngay, target: Level 3) như sau:

- **Phase 1 — Structured Logging + Correlation ID** (chi phí triển khai thấp, tác động cao): Chuẩn hóa JSON logging cho các dịch vụ Java và zerolog field names cho Go, đồng thời bổ sung **X-Request-ID** tại Gateway để lan truyền qua các HTTP headers hay Kafka. Giá trị nhận được ngay lập tức là khả năng gỡ rối chéo dịch vụ chỉ qua việc tìm kiếm một traceId duy nhất.
- **Phase 2 — Centralized Log Aggregation** (chi phí triển khai trung bình): Triển khai cụm Promtail, Loki và Grafana để thu thập log từ cả hệ thống LMS core lẫn DevOps Lab. Việc cấu hình pipeline thu thập log tập trung sẽ chấm dứt ngay thói quen SSH vào từng container để đọc log thủ công.
- **Phase 3 — Distributed Tracing** (chi phí triển khai trung bình): Tích hợp Micrometer Tracing và OpenTelemetry vào mỗi dịch vụ, cùng với Jaeger làm backend, cho phép chúng ta nhìn rõ chính xác điểm nghẽn mạng — liệu sự chậm trễ đến từ Judge Service, Kafka hay Database.
- **Phase 4 — Business Metrics + Alerting** (chi phí triển khai trung bình): Thêm Prometheus để thu thập số liệu và Grafana để trực quan hóa, từ đó thiết lập các cảnh báo dựa trên SLO thực tế như submission latency. Các cảnh báo sẽ được phân cấp hợp lý để gửi thông báo tức thời nếu Judge Service ngừng hoạt động, nhưng chỉ hiển thị độ trễ bất thường dưới dạng cảnh báo nhẹ.

Những cạm bẫy về Observability. Hệ thống giám sát nếu được thiết lập sai cách sẽ tự sinh thêm nhiều dữ liệu (noise), gây lãng phí tài nguyên và làm giảm độ nhạy bén của đội ngũ vận hành.

Mỗi điểm theo dõi thêm vào đều tiêu tốn tài nguyên — độ chi tiết của dữ liệu phải được cân với chi phí vận hành. Dưới đây là những sai lầm phổ biến nhất khi triển khai Observability:

1. **Log không cấu trúc, thiếu correlation ID** — Mỗi service log free-text riêng, không có ID xuyên suốt một request; khớp log bằng timestamp sai lệch ngay khi có xử lý đồng thời → debug như “mò kim đáy bể”. *Phòng tránh:* structured logging (JSON) + correlation/trace ID khởi tạo tại Gateway và lan truyền qua mọi service.
2. **Logging quá nhiều hoặc quá ít** — Ghi toàn bộ request body làm lộ dữ liệu nhạy cảm và cạn kiệt lưu trữ; chỉ ghi log lỗi lại mất bối cảnh khi debug. *Phòng tránh:* INFO cho sự kiện nghiệp vụ, DEBUG cho chi tiết kỹ thuật; tuyệt đối không ghi mật khẩu hay PII.
3. **Alert trên symptom thay vì SLO** — Cảnh báo cho mọi metric kỹ thuật rời rạc (CPU, một lỗi lẻ) sinh hàng chục thông báo mỗi ngày. Hậu quả: alert fatigue — đội ngũ “miễn nhiễm” với cảnh báo, bỏ lỡ sự cố thật. *Phòng tránh:* cảnh báo dựa trên SLO/error budget gắn với trải nghiệm người dùng, phân loại mức độ nghiêm trọng rõ ràng.
4. **Cardinality explosion trong metrics** — Gắn giá trị biến thiên cao (userId, submissionId, email) làm nhãn cho Prometheus. Hậu quả: số time-series bùng nổ → quá tải hệ thống metrics, tốn chi phí, truy vấn chậm. *Phòng tránh:* chỉ dùng label cardinality thấp (status, method, service); để chi tiết cho logs/traces.
5. **Dùng tracing thay cho logging** — Tracing cho biết request đi đâu và tồn bao lâu, nhưng không giải thích tại sao thất bại. *Phòng tránh:* kết hợp cả hai — tracing cho cái nhìn tổng quan, logging làm rõ chi tiết.
6. **Liveness probe “đào” quá sâu** — Health check của liveness gọi tới DB hoặc downstream. Hậu quả: khi cơ sở dữ liệu xử lý chậm, Kubernetes nhận định sai rằng pod đã chết và kill tiến trình hàng loạt → sự cố lan rộng. *Phòng tránh:* liveness chỉ kiểm tra bản thân tiến trình (shallow); đưa kiểm tra dependency vào readiness.

7. **Bỏ qua observability vì “nhóm nhỏ”** — Ban đầu `docker logs` là đủ, nhưng khi hệ thống mở rộng, đọc log thủ công từ hàng chục container tiềm ẩn nhiều rủi ro vận hành. *Phòng tránh*: bắt đầu ngay với structured logging + correlation ID — bước tốn ít công sức nhất nhưng giá trị nhất.

Interactive Demo

Xem minh họa động tại companion web:

Truy vết phân tán (Distributed Tracing) – – distributed-tracing.html

Tập trung hóa cấu hình (Config Server) – – config-server.html

11.10. Tổng kết

Observability là **điều kiện tiên quyết** để vận hành microservices — không phải tính năng bổ sung. Trong monolith, một log file và một debugger đủ để tìm bug. Trong microservices, request đi qua nhiều services, failure modes không dự đoán được — cần công cụ cho phép *hỏi câu hỏi mới* về hành vi hệ thống mà không cần deploy code mới.

Ba trụ cột — Logs, Metrics, Traces — cung cấp thông tin đa chiều: metrics phát hiện vấn đề (latency spike), traces xác định vị trí (Judge Service), logs giải thích nguyên nhân (SQL timeout). Observability Maturity Model giúp nhóm xác định *mức độ cần thiết* — không phải hệ thống nào cũng cần Level 5. Với LMS, Level 3 (Proactive) là mục tiêu hợp lý.

Structured logging và Correlation ID là nền tảng — chi phí triển khai thấp nhất, tác động cao nhất. SLI/SLO/SLA cung cấp framework để chuyển từ “monitoring metrics” sang “measuring service quality” — đặc biệt quan trọng cho contest mode của LMS khi downtime ảnh hưởng trực tiếp đến trải nghiệm học tập.

Error handling nhất quán — centralized exception handler + error code taxonomy — phục vụ cả client (UX tốt hơn) và đội ngũ vận hành (metrics theo error code category). User tracking qua Kafka pipeline mở rộng observability từ *hệ thống* sang *hành vi học tập* — giá trị riêng mà domain giáo dục khai thác từ observability.

Phân tích KBM cho thấy observability đã có nhiều mảnh đúng hướng nhưng chưa thành một contract end-to-end. Migration path rõ ràng: structured logs → correlation IDs → centralized aggregation → tracing → metrics + SLOs. Mỗi giai đoạn mang lại giá trị ngay, không cần đợi “hoàn thiện toàn bộ” mới bắt đầu. Ở Chương 12, chúng ta sẽ chuyển sang **Triển khai và DevOps** — containerization, Docker Compose, CI/CD pipelines, và cách deploy hệ thống microservices hoàn chỉnh.

Bài tập Chương 11

Xem Phụ lục Bài tập để luyện tập:

11-1 – Debugging: Latency Spike Mystery ([L3])

11-2 – Observability Stack: Build vs Buy vs Managed (Trade-off Analysis [L2])

11.11. Đọc thêm

Sách tham khảo chính:

- [2a] Chris Richardson, *Microservices Patterns* 1st Ed. — Ch.11: Developing Production-Ready Services — Health check API, Log aggregation, Distributed tracing, Application metrics

2. [4a] Sam Newman, *Building Microservices* — Ch.8: Monitoring — Logs, Metric tracking, Synthetic monitoring, Correlation IDs

Sách bổ trợ:

3. [5] Hugo Rocha, *Practical Event-Driven MS Architecture* — Distributed Tracing, Domain Segregation
4. [3] Ronnie Mitra & Irakli Nadareishvili, *Microservices: Up and Running* — Ch.9: Monitoring infrastructure
5. [4b] Sam Newman, *Monolith to Microservices* — Reporting, Monitoring and Troubleshooting during migration

Nguồn trực tuyến:

- OpenTelemetry official docs — opentelemetry.io
- Micrometer docs — micrometer.io (Spring Boot observability)
- Grafana Loki — grafana.com/oss/loki (lightweight log aggregation)
- Jaeger — jaegertracing.io (distributed tracing)
- Grafana Tempo — grafana.com/oss/tempo (tracing backend cùng hệ sinh thái Grafana, đang dần thay thế Jaeger trong các stack mới)
- Brendan Gregg, “USE Method” — brendangregg.com/usemethod.html
- Tom Wilkie, “RED Method” — grafana.com/blog/2018/08/02/the-red-method
- Google SRE Book, “Service Level Objectives” — sre.google/sre-book/service-level-objectives

Tài liệu tham khảo

- Newman, S. (2019). *Monolith to Microservices: Evolutionary Patterns to Transform Your Monolith*. O'Reilly Media.